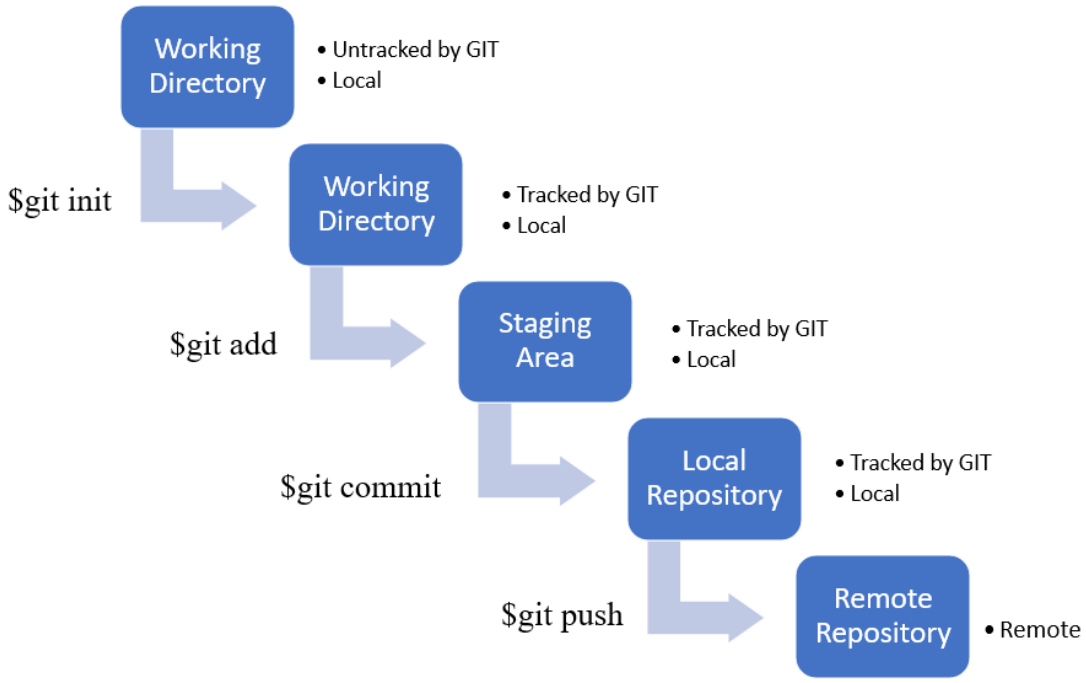
Git and Git Hub

Git is a version-control system for tracking changes in computer files and coordinating work on those files among multiple files.

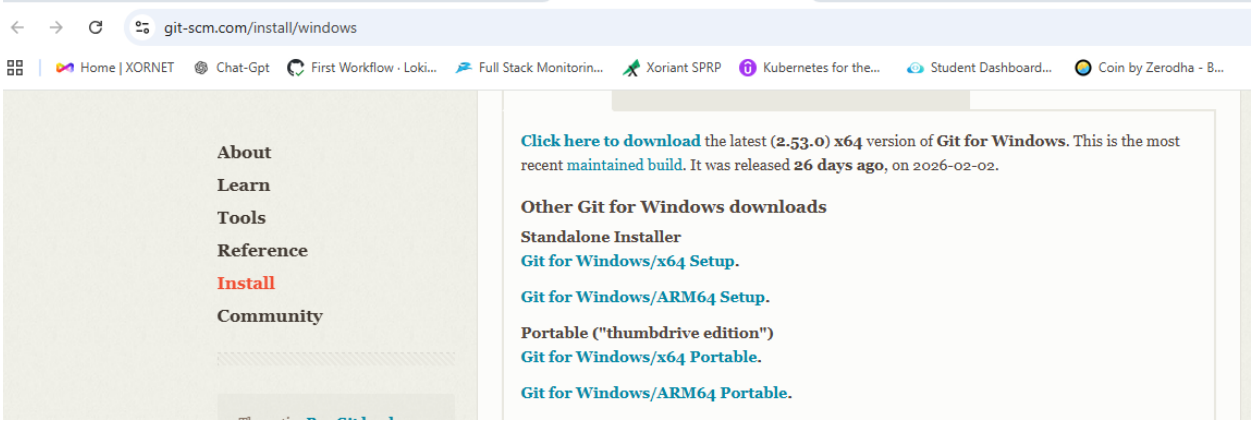
GIT LIFE CYCLE



Installing Git

https://git-scm.com/install/windows

Install based on the requirements



2 Also can be installed from AWS EC2 instance as well**.**

Need to have an AWS account and create Ec2 instance connect to Ec2 instance from any Ubuntu, Amazon Linux etc.,

Ubuntu

sudo apt update

sudo apt install git -y

git –version

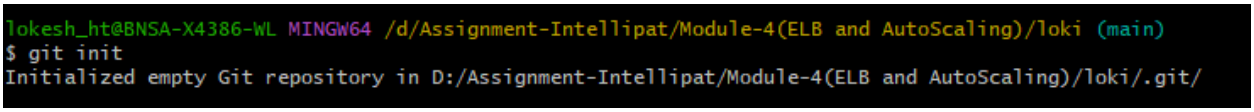
Amazon Linux 2

sudo yum update -y
sudo yum install git -y

git --version

Git Commands

1. **Git init -----** Initialize a repository



2. **Git status ---** Tracking the files

```
lokesh_ht@BNSA-X4386-WL MINGW64 /d/Assignment-Intellipat/Module-4(ELB and AutoScaling)/loki (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        a1.txt

nothing added to commit but untracked files present (use "git add" to track)
```

3. **Git add –** Staging the files

Git add . **---** all files in the directory

Git add <filename> -- specific files

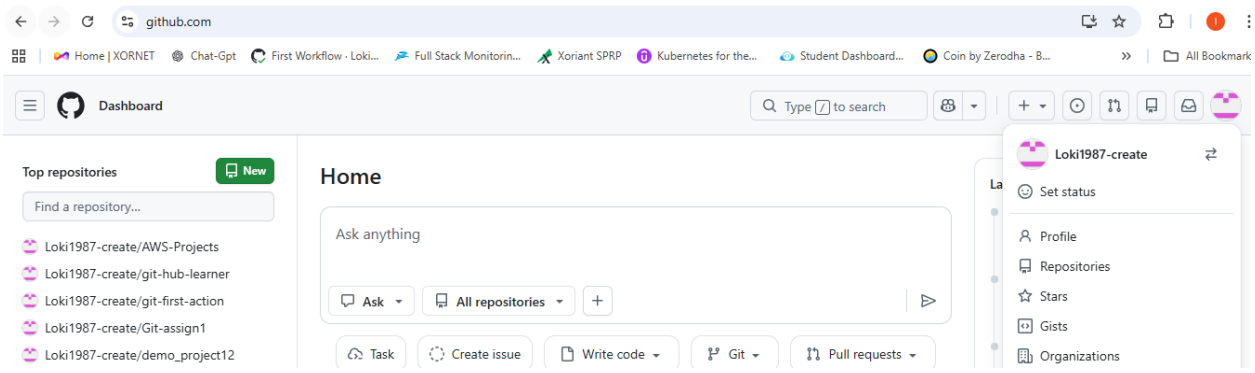
```
lokesh_ht@BNSA-X4386-WL MINGW64 /d/Assignment-Intellipat/Module-4(ELB and AutoScaling)/loki (master)
$ git add .

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Assignment-Intellipat/Module-4(ELB and AutoScaling)/loki (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   a1.txt
```

4. **Git commit -m “message”**

```
lokesh_ht@BNSA-X4386-WL MINGW64 /d/Assignment-Intellipat/Module-4(ELB and AutoScaling)/loki (master)
$ git commit -m "first commit"
[master (root-commit) 3366423] first commit
 1 file changed, 2 insertions(+)
 create mode 100644 a1.txt
```

Git-Hub

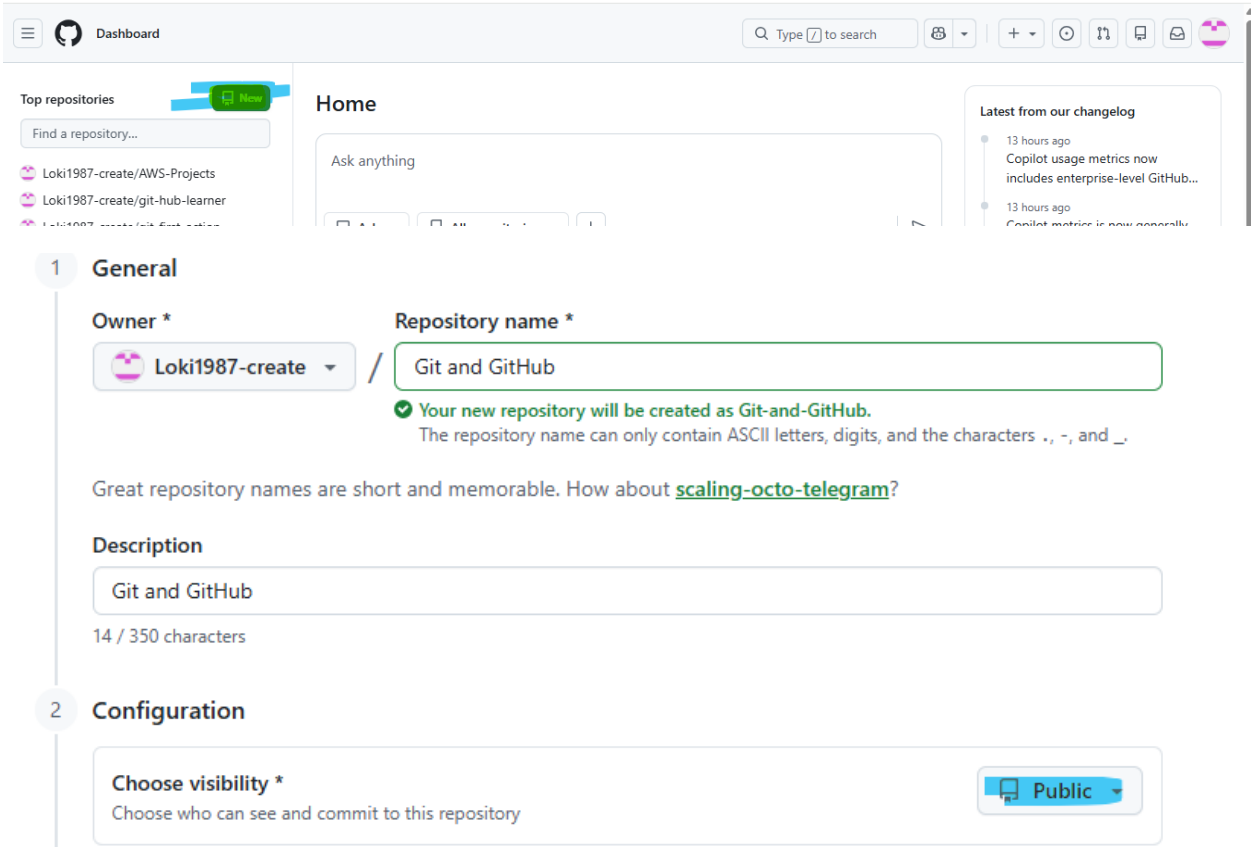
Git-Hub is an online platform used to store, manage, and share code using **Git** (a version control system).



Go to git-hub.com and create a user, normal process would not cost.

Command to move the committed codes to remote repository or **Git-Hub**

1. Create a new repository on Git-Hub



Choose the visibility, if this is public then anyone can access this code.

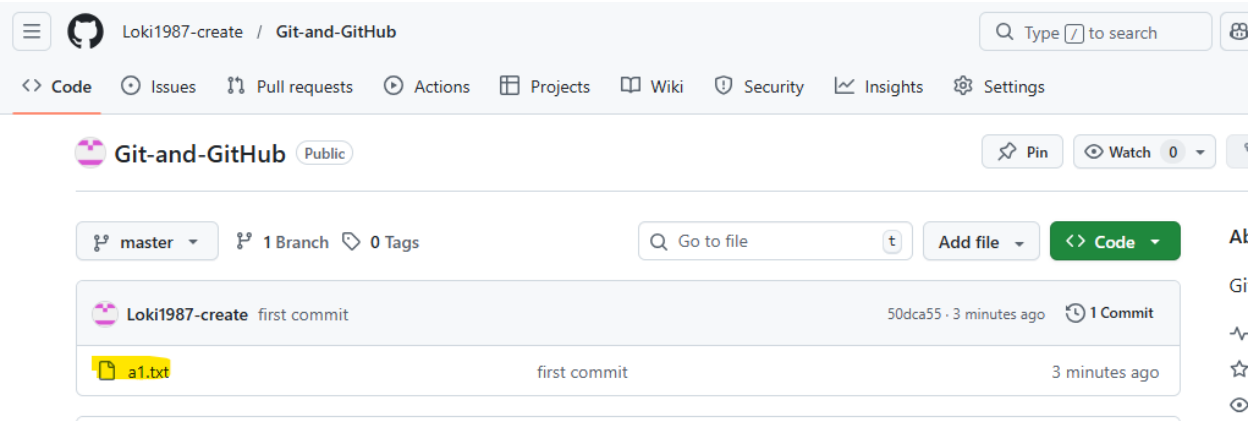
5. **git remote add origin https://github.com/Loki1987-create/Git-and-GitHub.git**

Loki1987-create --- > Username of git reposiroty

Git-and-GitHub.git -- > Name of the repository on Git-Hub

6. **git push -u origin master -- >** push the committed code to remote repo

```
lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$ git remote add origin https://github.com/Loki1987-create/Git-and-GitHub.git
error: remote origin already exists.

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 226 bytes | 226.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Loki1987-create/Git-and-GitHub.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$
```



Git clone

7. **git clone <** https://github.com/Loki1987-create/Git-and-GitHub**> →**

The remote repo URL of the, the command would

It automatically:

1. Creates a folder with the repository name
2. Downloads all files
3. Downloads commit history
4. Connects to remote as origin

Git Branch

8. **git branch <branch-name>** → creates a new branch

9. **git branch** → lists the branches available

```
lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$ git branch new-branch

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$ git branch
* master
  new-branch

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$
```

Master highlighted in green represents that you are in master branch

10. **git checkout <branch name>** → to switch to another branch
11. **git switch <branch-name>** → to switch to another branch
12. **git branch -d <branch-name>→** to delete the branch

```
lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$ git branch new-branch

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$ git branch
* master
  new-branch

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$ git checkout new-branch
Switched to branch 'new-branch'

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (new-branch)
$ git branch
  master
* new-branch

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (new-branch)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$ git branch -d new-branch
Deleted branch new-branch (was 50dca55).

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$ git branch
* master

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (master)
$
```

MERGINIG BRANCHES

Once the Developer has finished his code/features on his branch, the code will have to be combined with the master branch.

13. **git merge <source-branch>** → Merge the changes with the source branch after the changes.

14. **git log** → shows the history of commits in your repository.

```
lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (test-branch)
$ git log
commit d5b9035be35190950d2954fbd46bc2dd92159d4a (HEAD -> test-branch)
Author: Lokesh-Learner <lokeshht.2007@gmail.com>
Date:   Sat Feb 28 14:52:57 2026 +0530

    code.txt

commit 50dca55cb7e5d71edc93d9b0baa6e1b5c11a1a90 (origin/master, master)
Author: Lokesh-Learner <lokeshht.2007@gmail.com>
Date:   Sat Feb 28 12:52:01 2026 +0530

    first commit
```

15. **git log -- graph**

This gives:

- One-line commits
- Visual branch tree
- All branches

```
lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (test-branch)
$ git log
commit d5b9035be35190950d2954fbd46bc2dd92159d4a (HEAD -> test-branch)
Author: Lokesh-Learner <lokeshht.2007@gmail.com>
Date:   Sat Feb 28 14:52:57 2026 +0530

    code.txt

commit 50dca55cb7e5d71edc93d9b0baa6e1b5c11a1a90 (origin/master, master)
Author: Lokesh-Learner <lokeshht.2007@gmail.com>
Date:   Sat Feb 28 12:52:01 2026 +0530

    first commit

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (test-branch)
```

16. **git log --graph --pretty=oneline**

```
lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (test-branch)
$ git log --graph --pretty=oneline
* d5b9035be35190950d2954fbd46bc2dd92159d4a (HEAD -> test-branch) code.txt
* 50dca55cb7e5d71edc93d9b0baa6e1b5c11a1a90 (origin/master, master) first commit

lokesh_ht@BNSA-X4386-WL MINGW64 /d/Devops/Git (test-branch)
```

17. **git rebase**

*Both Git **merge** and **rebase** are used to combine changes from one branch into another, but they work differently in how they handle commit history.*

Feature	Merge	Rebase
History	Non-linear	Linear
Merge commit	Yes	No
Safe for shared branches	Yes	No
Clean history	No	Yes

Git Merge

Plain text

```
A---B---C  main
     \
      D---E  feature
```

After merge:

Plain text

```
A---B---C-------F  main
     \          /
      D---E-----   feature
```

Git Rebase

```
Plain text

A---B---C  main
     \
      D---E  feature
```

After rebase:

```
Plain text

A---B---C---D'---E'  feature
```

Commits **D and E are recreated as new commits (D', E').**

Important Rule:

Never rebase a public/shared branch.
Because rebase rewrites history and can break others' work.

MERGE CONFLICT

A merge conflict in Git happens when Git cannot automatically combine changes from two branches because the same part of a file was modified differently.

18. git mergetool

Example Scenario

Step 1 – Original file

```
Plain text

Hello World
```

Step 2 – Two developers change it

Branch A

```
Plain text

Hello DevOps
```

Branch B

```
Plain text

Hello Kubernetes
```

Now when merging:

```bash
git merge branchB
```

Git detects a **merge conflict.**

How Git Shows a Conflict

The file will look like this:

```
<<<<<<< HEAD
Hello DevOps
=======
Hello Kubernetes
>>>>>>> branchB
```

Meaning:

Marker	Meaning
`<<<<<<< HEAD`	current branch code
`=======`	separator
`>>>>>>> branchB`	incoming branch code

Fix the Conflict

Example resolved version:

```
Hello DevOps and Kubernetes
```

Step 2 — Stage the file

```bash
git add filename
```

Step 3 — Complete the merge

```bash
git commit
```

Full Conflict Resolution Workflow

```bash
git pull
git merge feature-branch
# conflict occurs
# edit file manually
git add file
git commit
```

Forking in GitHub

A fork is a copy of repository, forking a repository allows you to freely experiment with changes without affecting the original project.

Git reset

18. git reset

git reset in Git is used to undo commits or move the branch pointer to a previous commit.
It can also unstage files or delete commits depending on the option used.

git reset [option] <commit>

git reset HEAD~1

This moves the branch one commit back.

Area	Result
Commit history	Last commit removed
Staging area	Changes kept
Working directory	Changes kept

Three types of Git reset

1. git reset --soft HEAD~1 -- This moves branch **one commit back.**
2. git reset --mixed HEAD~1 -- Files become **unstaged but still exist**.
3. git reset --hard HEAD~1 -- **Dangerous** because changes are permanently removed.